

Java Backend Architecture

Interview Series

Chapter 16.3

Multi-Stage Builds

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

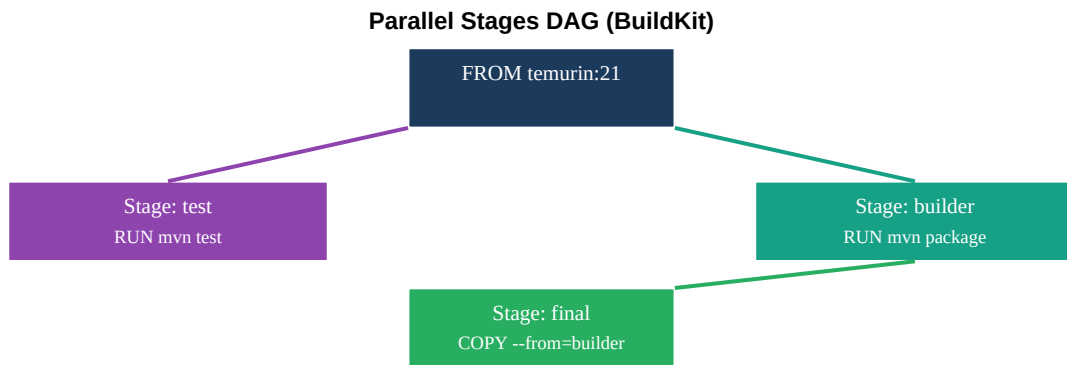
Multi-Stage Builds Overview

Multi-Stage Build Flow & Size Reduction



BuildKit Cache Mounts for Maven

Parallel Stages DAG



BuildKit analyzes the dependency graph between stages. Stages that do not depend on each other are executed in parallel. Use `--target` to build only a specific stage — useful in CI/CD to run tests separately from building the production image.

--target Flag Usage

```
# Run only up to the test stage (in CI) docker build --target test -t myapp:test . # Build only the builder stage docker build --target builder -t myapp:builder . # Build final production image (default) docker build -t myapp:1.0 .
```

COPY --from Patterns

```
# Copy from named stage COPY --from=builder /app/target/*.jar /app/app.jar # Copy from stage by index (0-based) COPY --from=0 /app/target/*.jar /app/app.jar # Copy from external image (not a stage) COPY --from=nginx:alpine /etc/nginx/nginx.conf /etc/nginx/ # Copy with ownership COPY --from=builder --chown=nonroot:nonroot /app/app.jar /app/
```

JLink Custom JRE

JLink (Java 9+) creates a minimal custom JRE containing only the modules your application needs. Combined with multi-stage builds, this reduces the JRE from ~200MB to 50-100MB depending on required modules.

```
FROM eclipse-temurin:21 AS jlink-builder # Analyze required modules RUN jdeps --ignore-missing-deps \ --print-module-deps \ --multi-release 21 \ /app/app.jar > modules.txt # Build custom JRE RUN jlink --add-modules $(cat modules.txt) \ --strip-debug \ --no-man-pages \ --no-header-files \ --compress=2 \ --output /custom-jre FROM debian:bookworm-slim COPY --from=jlink-builder /custom-jre /opt/jre ENV PATH="/opt/jre/bin:$PATH" COPY --from=builder /app/app.jar /app/ ENTRYPOINT ["/opt/jre/bin/java", "-jar", "/app/app.jar"]
```

Size Comparison Table

Approach	Image Size	Notes
Single stage (JDK + Maven)	~600MB	Development only; never use in production
Multi-stage JRE (ubuntu)	~250MB	Common production baseline
Multi-stage JRE (Alpine)	~120MB	Smaller but Alpine glibc differences
Multi-stage + distroless	~90MB	Recommended for production security
Multi-stage + JLink + slim	~75MB	Best size; requires module analysis
Multi-stage + JLink + distroless	~55MB	Minimal size + max security

Interview Q&A; – Multi-Stage Builds

Q1. What are multi-stage builds and what problem do they solve?

Multi-stage builds use multiple FROM instructions in a single Dockerfile. Each FROM starts a new build stage with its own filesystem. You can selectively COPY files between stages with COPY --from=stagenamename. This solves the problem of build tool contamination — Maven, JDK, test frameworks, and source code are kept in the builder stage; only the compiled JAR is copied to the minimal runtime stage. Result: 600MB builder image produces a 90-200MB production image.

Q2. How does COPY --from work in a multi-stage build?

COPY --from=stagenamename copies files from a named stage or another image into the current stage. Syntax: COPY --from=builder /app/target/app.jar /app/app.jar. You can reference stages by name (AS builder) or by 0-based index (--from=0). You can also reference external images: COPY --from=nginx:alpine /etc/nginx/nginx.conf /etc/nginx/. Only explicitly copied files appear in the final image — everything else in intermediate stages is discarded.

Q3. What is the --target flag in docker build?

The --target flag stops the build at a specified stage: docker build --target builder . This is useful in CI/CD to separate build phases: run docker build --target test to execute tests without producing the production image (fails fast if tests fail), and docker build --target final for production. It enables reuse of intermediate stages for different purposes without duplicating Dockerfile logic.

Q4. How does BuildKit enable parallel stage execution?

BuildKit analyzes the DAG (directed acyclic graph) of stage dependencies. Stages that don't depend on each other run in parallel. For example: a test stage and a documentation stage both derived from the base stage run simultaneously, reducing total build time. Legacy Docker builder executed all stages serially. Enable BuildKit: DOCKER_BUILDKIT=1 or use docker buildx build (default since Docker 23). Parallel execution is most beneficial in complex multi-service monorepos.

Q5. What are BuildKit cache mounts and how do they improve Java build speed?

BuildKit cache mounts (--mount=type=cache,target=/root/.m2) provide a directory that persists between builds but is NOT stored in image layers. For Maven, this means the entire ~/.m2 repository cache is reused across builds on the same host. First build: 4 minutes (downloads all deps). Subsequent builds: 30-60 seconds (uses cache). Unlike Docker layer cache, BuildKit cache mounts are not invalidated by source code changes — only the actual Maven download cache matters.

Q6. Why use distroless as the final stage base image?

Distroless images contain only the language runtime — no shell, no package manager, no OS utilities. For Java: gcr.io/distroless/java21-debian12 contains only the JRE and necessary Debian libs. Benefits: (1) Minimal attack surface — attackers cannot run shell commands even if they achieve RCE. (2) Smaller image (~90MB). (3) Fewer CVEs to patch. (4) Passes stricter security scanners. Drawback: no debug shell. Use :debug variant for troubleshooting — it adds busybox with sh.

Q7. Explain Spring Boot layered JAR extraction in multi-stage builds.

Spring Boot 2.3+ supports layered JARs. java -Djarmode=layertools -jar app.jar extract splits the JAR into 4 directories: dependencies/ (third-party libs), spring-boot-loader/ (Spring's JAR loader), snapshot-dependencies/ (SNAPSHOT versions), application/ (your code). In multi-stage builds, each directory becomes a separate COPY instruction and thus a separate Docker layer. On redeploy, only the application/ layer (few KB) changes — dependencies/ layer (100MB+) is reused, saving significant bandwidth.

Q8. What is JLink and how does it reduce Docker image size?

JLink is a Java tool (since Java 9) that creates a custom, minimal JRE containing only the modules your application needs. Run `jdeps --print-module-deps app.jar` to determine required modules, then `jlink --add-modules module.list --output /custom-jre`. A typical Spring Boot app needs: `java.base`, `java.net.http`, `java.sql`, `java.naming`, `java.desktop` (for some libs), reducing JRE from 200MB to 50-80MB. Combined with multi-stage builds and distroless, final image can be under 60MB.

Q9. How do you handle Maven settings.xml with private repositories in multi-stage builds?

Use BuildKit secret mounts: `RUN --mount=type=secret,id=maven_settings,dst=/root/.m2/settings.xml mvn package`. Pass at build time: `docker buildx build --secret id=maven_settings,src=~/.m2/settings.xml`. The settings.xml with Artifactory credentials is available only during that RUN step — it never appears in any image layer or docker history. Alternative: use ARG for the Artifactory token (less secure, appears in history), or inject credentials via environment variables at build time.

Q10. What is the FROM scratch base image and when is it used?

FROM scratch starts with a completely empty filesystem — no OS, no shell, nothing. Used for: (1) Statically compiled binaries (Go, Rust) that have no dynamic library dependencies. (2) Building other base images (like Alpine, Debian minimal). For Java applications, scratch is not practical because the JVM requires glibc and system libraries. Use distroless instead for Java. FROM scratch + static binary is popular for network proxies, sidecars, and utilities that don't need a JVM.

Q11. How do multi-stage builds work with Docker layer caching?

Each stage maintains its own layer cache. When you docker build, Docker checks if each layer in each stage is cached. COPY pom.xml + RUN mvn dependency:go-offline in the builder stage are cached until pom.xml changes. COPY src/ is always near the end, invalidating only the final few layers. The final stage cache is separate from the builder stage. BuildKit is smarter: it only rebuilds stages that are actually needed for the requested target, skipping unused stages entirely.

Q12. Can you COPY from a stage that was skipped with --target?

When using --target, Docker builds up to and including the target stage. Stages after the target are not built. If the target stage doesn't depend on a later stage, that stage is skipped. You cannot COPY --from a stage that hasn't been built. However, BuildKit only builds stages reachable from the target — stages not in the dependency chain are skipped even without --target. This is the lazy evaluation feature of BuildKit that legacy builder lacked.

Q13. How do you implement a test stage that fails the build if tests fail?

Create a dedicated test stage: FROM builder AS test; RUN mvn test. In CI, run: `docker build --target test --tag myapp:test`. If mvn test exits non-zero, docker build fails and the CI pipeline stops. The production build: `docker build --tag myapp:prod`. skips directly to the final stage (build cache reuses the test layer). This ensures tests run before production images are created without the test JARs or test dependencies ending up in the production image.

Q14. What are the best practices for naming stages in a multi-stage Dockerfile?

Use descriptive names with AS: FROM maven:3.9 AS dependencies, FROM maven:3.9 AS builder, FROM eclipse-temurin:21-jre AS runtime, FROM runtime AS debug. Good names: base, builder, test, integration-test, final, runtime, dev, prod. Avoid generic names like build1, stage2. Named stages are referenced in COPY --from=stagename and --target=stagename. Document the purpose of each stage with comments. Keep stage names consistent across Dockerfiles in a project.

Q15. How do multi-stage builds affect CI/CD pipeline performance?

Multi-stage builds improve CI/CD performance by: (1) Layer caching — only changed layers rebuild, reducing build time from minutes to seconds. (2) Parallel stages — independent stages run concurrently. (3) --target optimization — test and build stages can run in parallel CI jobs using the same Dockerfile. (4) Registry cache (--cache-from) — pull cached layers from registry in CI where local cache doesn't persist. (5) Smaller final images mean faster push/pull. Typical result: 5-minute build reduced to 45 seconds on cache hit.

Q16. What is the syntax=docker/dockerfile:1 directive?

The # syntax=docker/dockerfile:1 directive at the top of a Dockerfile specifies the BuildKit frontend syntax. This enables the latest Dockerfile features (heredoc syntax, cache mounts, secret mounts, SSH mounts) regardless of the Docker version installed. Docker automatically pulls the frontend from Docker Hub when building. Use # syntax=docker/dockerfile:1.4 or later for heredoc support. This is a BuildKit-specific feature — it's ignored by the legacy builder.

Q17. How do you copy files from multiple stages in a final stage?

A single stage can COPY from multiple previous stages: COPY --from=builder /app/app.jar /app/ and COPY --from=config-stage /app/config/ /app/config/ and COPY --from=frontend-stage /dist/ /app/static/. This assembles a final image from multiple specialized build stages. Common pattern for full-stack apps: separate stage builds the React frontend, another builds the Spring Boot backend, and the final stage combines both into a single deployable image.

Q18. What is --build-arg in multi-stage builds and when does it apply?

ARG declared before all FROM instructions is global and applies to all stages' FROM instructions (for dynamic base images). ARG declared within a stage applies only to that stage. To use a global ARG inside a stage, you must re-declare it: ARG VERSION at the top level, then ARG VERSION again inside the stage. Useful for: FROM temurin:\${JAVA_VERSION:-21}-jre-jammy to parameterize the Java version. With multi-stage, this allows building different Java versions without editing the Dockerfile.

Q19. Explain Docker's registry cache (--cache-from) in CI/CD.

--cache-from pulls a previously built image to use as a layer cache source for the current build. In CI where local Docker cache doesn't persist between jobs: docker pull myapp:latest || true; docker buildx build --cache-from type=registry,ref=myregistry/myapp:latest --cache-to type=inline . By pushing inline cache metadata to the registry, subsequent CI runs pull only the changed layers. More efficient: use external cache backends (--cache-to type=registry,mode=max) to cache all layers including intermediate stages.

Q20. When should you NOT use multi-stage builds?

Multi-stage builds add complexity. They may not be worth it when: (1) The application already uses a minimal runtime (e.g. a Go binary). (2) The Dockerfile is simple and the size difference is negligible. (3) Build environments are tightly controlled (internal only, no registry size costs). (4) Team is not familiar with Docker and the complexity hinders maintenance. (5) The build tool and runtime have the same base (some Python/Node.js apps). Always measure the size benefit vs. complexity trade-off.

Q21. How do multi-stage builds help with secret management?

Multi-stage builds prevent secrets from leaking into final images. Pattern: in the builder stage, use BuildKit secret mounts (--mount=type=secret) for credentials like Maven settings.xml, npm tokens, or SSH keys. These secrets are available only during that RUN step. The final stage only receives the compiled artifact via COPY --from= — no secrets, no credentials. Combined with .dockerignore (exclude .env files), multi-stage builds ensure production images contain no build-time credentials.